

Antoine Lecomte

Lab 2b : Kubernetes part 2

```
user@deb:~$ minikube start --driver=docker --cni=calico cpus=4 --memory=4096
🐳 minikube v1.37.0 on Debian 13.1 (vbox/amd64)
🔧 Using the docker driver based on user configuration

💡 The requested memory allocation of 4096MiB does not leave room for system overhead (total system memory: 4923MiB). You may face stability issues.
💡 Suggestion: Start minikube with less memory allocated: 'minikube start --memory=4096mb'

🔥 Using Docker driver with root privileges
👍 Starting "minikube" primary control-plane node in "minikube" cluster
🚢 Pulling base image v0.0.48 ...
🔥 Creating docker container (CPUs=2, Memory=4096MB) ...

💡 Docker is nearly out of disk space, which may cause deployments to fail! (85% of capacity). You can pass '--force' to skip this check.
💡 Suggestion:

    Try one or more of the following to free up space on the device:

    1. Run "docker system prune" to remove unused Docker data (optionally with "-a")
    2. Increase the storage allocated to Docker for Desktop by clicking on:
       Docker icon > Preferences > Resources > Disk Image Size
    3. Run "minikube ssh -- docker system prune" if using the Docker container runtime
    Related issue: https://github.com/kubernetes/minikube/issues/9024

🚢 Preparing Kubernetes v1.34.0 on Docker 28.4.0 ...
🔧 Configuring Calico (Container Networking Interface) ...
🔍 Verifying Kubernetes components...
   ■ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟 Enabled addons: storage-provisioner, default-storageclass

❗ /usr/bin/kubectl is version 1.32.3, which may have incompatibilities with Kubernetes 1.34.0.
   ■ Want kubectl v1.34.0? Try 'minikube kubectl -- get pods -A'
👤 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
user@deb:~$ minikube addons enable ingress
💡 ingress is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
   ■ Using image registry.k8s.io/ingress-nginx/controller:v1.13.2
   ■ Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
   ■ Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
🔍 Verifying ingress addon...
🌟 The 'ingress' addon is enabled
```

We start minikube, create and apply an Ingress resource to begin.

The host is wiki.cluster.org and is linked to the wiki-svc service, which is running on port 3000.

```

user@deb:~/Documents/lab2$ kubectl apply -f wikijs-ingress.yaml
ingress.networking.k8s.io/wikijs-ingress created
user@deb:~/Documents/lab2$ kubectl get ingress
NAME                CLASS    HOSTS                ADDRESS        PORTS    AGE
wikijs-ingress      nginx    wikijs.cluster.org  192.168.49.2   80       8s
user@deb:~/Documents/lab2$ minikube tunnel
Status:
  machine: minikube
  pid: 25963
  route: 10.96.0.0/12 -> 192.168.49.2
  minikube: Running
  services: []
  errors:
    minikube: no errors
    router: no errors
    loadbalancer emulator: no errors

```

We know the address of the wiki.

By trying to go to <http://127.0.0.1/>, an error occurs. The website is unreachable. We're not seeing the Wikijs page because there is no local entry for this domain name system.

By running « minikube ip » or directly on the previous screen with the « minikube tunnel » command, we see that the address to assign to this DNS is 192.168.49.2.

Then, we modify the file `/etc/hosts` by adding the line as follows :

```
192.168.49.2  wikijs.cluster.org
```

Now, we still have no access to the wiki with <http://192.168.49.2/> but the Ingress NGINX send a request to the wiki-svc service on the port 3000. We now get the error 404 (not found), it was expected.

It works with <http://wikijs.cluster.org/>.

We modify the file wikijs-ingress.yaml with the new http rule as follows :

```
GNU nano 8.4 wikijs-ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: wikijs-ingress
spec:
  rules:
  - host: wikijs.cluster.org
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: wiki-svc
            port:
              number: 3000
  - http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: wiki-svc
            port:
              number: 3000
```

```
user@deb:~/Documents/lab2$ kubectl apply -f wikijs-ingress.yaml
ingress.networking.k8s.io/wikijs-ingress configured
user@deb:~/Documents/lab2$
```

And everything works if the yaml files used for the wiki-svc service are applied. Else, the error 503 (Service Temporarily Unavailable) occurs.

```
kubectl apply -f postgres.yaml
```

```
kubectl apply -f deployment.yaml
```

```
kubectl apply -f service.yaml
```

```
kubectl apply -f wikijs-ingress.yaml
```

```
user@deb:~/Documents/lab2$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
postgres-5d45787877-cqt5t	0/1	ContainerCreating	0	13s
wikijs-85c4ccd4cb-48vpb	0/1	ContainerCreating	0	81s
wikijs-85c4ccd4cb-ckqr2	0/1	ContainerCreating	0	81s
wikijs-85c4ccd4cb-nww94	0/1	ContainerCreating	0	81s

```

user@deb:~/Documents/lab2$ kubectl get svc
NAME            TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
kubernetes      ClusterIP     10.96.0.1       <none>           443/TCP          119m
postgres        ClusterIP     10.110.91.45    <none>           5432/TCP          3m55s
wiki-svc        NodePort      10.110.178.33   <none>           3000:30080/TCP   6m33s

```

Exposing both a NodePort and an Ingress for the same service is a redundant exposure. NodePort already opens a TCP port on each node while Ingress routes external http/s traffic via NGINX. There are two different external access paths to the same app which is confusing.

By doing that, we bypass Ingress logic : someone accesses the NodePort directly, the different protections (TLS termination, path routing, authentication...) are bypassed, so it defeats the purpose to use an Ingress.

Nodeports reserve specific port numbers and many of them clutters the node's port space and complicates maintenance or debugging.

Also, it's unclear : which entry point should be used by users ? The NodePort URL or the Ingress hostname. So the traffic monitoring and logging become fragmented between the two paths.

- ➔ We should use ClusterIP instead because the Ingress controller runs inside the cluster and can reach any ClusterIP service internally, there's no need to expose the service on a NodePort or on an external interface and it's mainly aligned with Kubernetes best practices for security.

Using a networking policy, we apply it and check Internet connectivity:

```

user@deb:~/Documents/lab2$ sudo nano policy.yaml
[sudo] password for user:
user@deb:~/Documents/lab2$ kubectl apply -f policy.yaml
networkpolicy.networking.k8s.io/default-deny created
user@deb:~/Documents/lab2$ ping google.com
PING google.com (142.250.179.78) 56(84) bytes of data.
64 bytes from par21s19-in-f14.1e100.net (142.250.179.78): icmp_seq=1 ttl=255 time=17.2 ms
64 bytes from par21s19-in-f14.1e100.net (142.250.179.78): icmp_seq=2 ttl=255 time=17.9 ms
64 bytes from par21s19-in-f14.1e100.net (142.250.179.78): icmp_seq=3 ttl=255 time=17.3 ms
64 bytes from par21s19-in-f14.1e100.net (142.250.179.78): icmp_seq=4 ttl=255 time=16.8 ms
64 bytes from par21s19-in-f14.1e100.net (142.250.179.78): icmp_seq=5 ttl=255 time=17.6 ms
^C
--- google.com ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 8072ms
rtt min/avg/max/mdev = 16.763/17.341/17.894/0.377 ms
user@deb:~/Documents/lab2$ kubectl run -it --rm testpod --image=busybox --restart=Never -- sh
ping google.com
If you don't see a command prompt, try pressing enter.
ping google.com
ping: bad address 'google.com'
/ #

```

```

user@deb:~/Documents/lab2$ kubectl create namespace test
namespace/test created
user@deb:~/Documents/lab2$ ls
deployment.yaml  policy.yaml  service.yaml  wikijs-clusterip.yaml  wikijs-ingress.yaml
'lab2 report.odt'  postgres.yaml  test-pod.yaml  wikijs-headless.yaml
user@deb:~/Documents/lab2$ kubectl get ns
NAME                STATUS  AGE
default             Active  4h32m
ingress-nginx       Active  4h28m
kube-node-lease     Active  4h32m
kube-public         Active  4h32m
kube-system         Active  4h32m
test                Active  105m
user@deb:~/Documents/lab2$ kubectl run ubuntu-test \
  --image=ubuntu:latest \
  --restart=Never \
  --namespace=test \
  --command -- sleep infinity
pod/ubuntu-test created

user@deb:~/Documents/lab2$ kubectl exec -it -n test ubuntu-test -- bash
ping google.com
root@ubuntu-test:/# ping google.com
bash: ping: command not found
root@ubuntu-test:/# █

```

The first command works : my Debian VM is sending ICMP packets, outside of Kubernetes.

Inside the container, we have the message « ping: bad address 'google.com' ».

The network policy blocks the connectivity.

Then, we remove the policy with « kubectl delete -f policy.yaml ».

```

user@deb:~/Documents/lab2$ kubectl delete -f policy.yaml
networkpolicy.networking.k8s.io "default-deny" deleted

```

The new network policy has to allow Wikijs to receive connections from every Pod but prevents it to initiate connections, and Wikijs should not be able to access the Internet.

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: wikijs-receive-only
  namespace: default
spec:
  podSelector:
    matchLabels:
      app: wikijs
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - {} # Permet toutes les connexions entrantes depuis tous les pods
  egress: [] # Bloque toutes les connexions sortantes

```

In this configuration :

- Pods → Wikijs : authorized
- Wikijs → Other pods : unauthorized
- Wikijs → Internet : unauthorized

```
user@deb:~/Documents/Lab2$ kubectl apply -f networkpolicy.yaml
kubectl get netpol -n default
networkpolicy.networking.k8s.io/wikijs-receive-only created
NAME                POD-SELECTOR  AGE
wikijs-receive-only  app=wikijs    0s
user@deb:~/Documents/Lab2$ kubectl get netpol -n default
kubectl describe netpol wikijs-receive-only -n default
NAME                POD-SELECTOR  AGE
wikijs-receive-only  app=wikijs    33s
Name:               wikijs-receive-only
Namespace:          default
Created on:          2025-11-11 19:05:27 +0100 CET
Labels:              <none>
Annotations:         <none>
Spec:
  PodSelector:      app=wikijs
  Allowing ingress traffic:
    To Port: <any> (traffic allowed to all ports)
    From: <any> (traffic not restricted by source)
  Allowing egress traffic:
    <none> (Selected pods are isolated for egress connectivity)
  Policy Types: Ingress, Egress
user@deb:~/Documents/Lab2$
```

After deleting minikube :

```
user@deb:~/Documents/Lab2$ minikube delete
🔥 Deleting "minikube" in docker ...
🔥 Deleting container "minikube" ...
🔥 Removing /home/user/.minikube/machines/minikube ...
💀 Removed all traces of the "minikube" cluster.
user@deb:~/Documents/Lab2$
```

We want to add a sidecar NGINX named logs-proxy that listens to the port 8080.

See below the updated wordpress.yaml file :

apiVersion: v1

kind: PersistentVolumeClaim

metadata:

name: mysql-pvc

spec:

accessModes:

- ReadWriteOnce

resources:

requests:

storage: 2Gi

apiVersion: apps/v1

kind: Deployment

metadata:

name: mysql

spec:

replicas: 1

selector:

matchLabels:

app: mysql

template:

metadata:

labels:

app: mysql

spec:

containers:

- name: mysql

image: mysql:5.7

ports:

- containerPort: 3306

env:

- name: MYSQL_ROOT_PASSWORD

value: "password"

- name: MYSQL_DATABASE

value: "wordpress"

volumeMounts:

- name: mysql-storage

mountPath: /var/lib/mysql

volumes:

- name: mysql-storage

persistentVolumeClaim:

claimName: mysql-pvc

apiVersion: v1

kind: Service

metadata:

name: mysql-service

spec:

selector:

app: mysql

ports:

- protocol: TCP

port: 3306

targetPort: 3306

apiVersion: v1

kind: PersistentVolumeClaim

metadata:

name: wordpress-pvc

spec:

accessModes:

- ReadWriteOnce

resources:

requests:

storage: 10Gi

apiVersion: v1

kind: ConfigMap

metadata:

name: nginx-config

data:

nginx.conf: |

events { }

http {

log_format json_combined escape=json

{

time_local "\$time_local",

remote_addr "\$remote_addr",

request "\$request",

status "\$status",

body_bytes_sent "\$body_bytes_sent",

http_referer "\$http_referer",

http_user_agent "\$http_user_agent"

};

access_log /dev/stdout json_combined;

server {

```
listen 8080;

location / {
    proxy_pass http://127.0.0.1:80;
}

}
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wordpress
spec:
  replicas: 1
  selector:
    matchLabels:
      app: wordpress
  template:
    metadata:
      labels:
        app: wordpress
    spec:
      containers:
        - name: wordpress
          image: wordpress:latest
          ports:
            - containerPort: 80
          env:
            - name: WORDPRESS_DB_HOST
```

```
    value: "mysql-service"
  - name: WORDPRESS_DB_USER
    value: "root"
  - name: WORDPRESS_DB_PASSWORD
    value: "password"
  - name: WORDPRESS_DB_NAME
    value: "wordpress"
volumeMounts:
  - name: wordpress-storage
    mountPath: /var/www/html
- name: logs-proxy
  image: nginx:latest
  ports:
    - containerPort: 8080
  volumeMounts:
    - name: nginx-config-volume
      mountPath: /etc/nginx/nginx.conf
      subPath: nginx.conf
volumes:
  - name: wordpress-storage
    persistentVolumeClaim:
      claimName: wordpress-pvc
  - name: nginx-config-volume
    configMap:
      name: nginx-config
      items:
        - key: nginx.conf
          path: nginx.conf
```

apiVersion: v1

kind: Service

metadata:

name: wordpress-service

spec:

selector:

app: wordpress

type: NodePort

ports:

- protocol: TCP

port: 80

targetPort: 8080

nodePort: 30080

Now, wordpress-service redirects to the Nginx proxy (sidecar) on the port 8080, which is exposed by logs-proxy in the nginx.conf file. Requests are in JSON format in STDOUT.

For testing :

kubectl apply -f wordpress.yaml

kubectl logs -f <wordpress-pod_name> -c logs-proxy

Using Istio as a service mesh :

istioctl install -f samples/bookinfo/demo-profile-no-gateways.yaml -y

kubectl label namespace default istio-injection=enabled

Istio in the default configuration allows communication between pods only inside the mesh, so between pods that have the Istio-proxy (sidecar) injected.

Istio injects in fact a proxy into each pod within namespaces labeled for automatic injection. All inbound and outbound traffic is intercepted by it. And if both source and destination pods are part of the mesh (if they use istio-proxy), traffic between them is allowed by default permissive policy (Istio's default).

1 : Check the namespaces labeled for Istio injection

```
kubectl get namespace --show-labels
```

2 : List Bookinfo pods to see which have sidecars

```
kubectl get pods -n default
```

3 : Exec into one pod to test pod-to-pod communication

```
kubectl exec -it <productpage-pod> -c productpage -- bash
```

Inside the pod, test connection to another service (« reviews » in the example)

```
curl http://reviews:9080/reviews/1
```

It also works on other available pods.

The sidecar containers (Envoy proxies) are installed inside each application pod that belongs to the mesh.

Each of the pods productpage, reviews, ratings, details, has two containers :

The application container and the Istio sidecar container (Istio-proxy).

In order to see both containers :

```
kubectl describe pod <productpage-pod>
```

To check what the proxy runs inside the pod :

```
kubectl exec -it <productpage-pod> -c istio-proxy -- ps aux
```